ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<1>>

Descripción



```
def req_1(catalog, anio, mes, descuento_min, descuento_max):
    def criterio(pedido_1, pedido_2):
        return pedido_1["Discount_Pct"] > pedido_2["Discount_Pct"]
    if pedido_1["Amount"] != pedido_2["Amount"]:
        return pedido_1["Amount"] > pedido_2["Amount"]
    return pedido_1["Order_ID"] <= pedido_2["Order_ID"]

start_time = get_time()

llave_mes = str(int(anio)).zfill(4) + "-" + str(int(mes)).zfill(2)
pedidos_mes = mlp.get(catalog["OrdenMes"], llave_mes)

filtrados = lt.new_list()
total_amount = 0.0
suma_precio = 0.0
suma_cajas = 0

if pedidos_mes is not None:
    for i in range(lt.size(pedidos_mes)):
        pedido = lt.get_element(pedidos_mes, i)
        descuento = pedido["Discount_Pct"]
        if descuento >= descuento_min and descuento <= descuento_max:
            lt.add_last(filtrados, pedido)
            total_amount += pedido["Amount"]
            suma_precio += pedido["Price_per_Box"]
            suma_cajas += pedido["Boxes_Shipped"]

total_pedidos = lt.size(filtrados)
promedio_precio = 0.0
promedio_cajas = 0.0
if total_pedidos > 0:
    lt.merge_sort(filtrados, criterio)
    promedio_precio = suma_precio / total_pedidos
    promedio_cajas = suma_cajas / total_pedidos
```

```
    end_time = get_time()
    return (delta_time(start_time, end_time), total_pedidos, total_amount,
            promedio_precio, promedio_cajas, filtrados)
```

Consulta los pedidos de un mes específico cuyo descuento esté dentro de un rango dado, sin recorrer la lista principal. Durante la carga se construye la tabla de hash OrdenMes (Linear Probing), con llave "YYYY-MM" y valor un array_list con los pedidos de ese mes. Con el año y el mes recibidos se reconstruye la llave y se accede al mes en una sola operación. Sobre ese subconjunto se filtra por descuento acumulando en el mismo recorrido el monto total, la suma de precios y la suma de cajas, para no hacer pasadas adicionales. Los pedidos que cumplen se guardan en un array_list que se ordena con Merge Sort. Si el mes no existe, mlp.get retorna None y el resultado es vacío sin error.

Entrada	catalog, anio, mes, descuento min, descuento_max
Salidas	tiempo, total pedidos, total_amount, promedio precio, promedio cajas, filtrados.
Implementado (Sí/No)	Implementado por Juan Rojas

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Construir la llave "YYYY-MM" a partir del año y el mes	O(1)
Buscar la llave en la tabla de hash OrdenMes (mlp.get, Linear Probing)	O(1) promedio
Recorrer los m pedidos del mes y filtrar por el rango de descuento, acumulando monto total, suma de precios y suma de cajas	O(m)
Ordenar los k pedidos filtrados con Merge Sort	O(k log k)
Calcular los dos promedios a partir de los acumuladores	O(1)
TOTAL	***O(m + k log k)***

Análisis

Indexar por mes evita recorrer los 193.608 pedidos: cada mes hay aproximadamente 8.000 registros y la búsqueda es O(1), así que el costo depende del tamaño del mes y del resultado, no del archivo. Se usó Linear Probing porque solo hay 24 llaves (los meses de 2022 y 2023) sobre capacidad 211, con factor de carga real de 0.11 y colisiones casi nulas; Separate Chaining añadiría una lista encadenada por casilla sin necesidad. El ordenamiento es Merge Sort, O(k log k) garantizado y estable.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<6>>

Descripción

```python
def req_6(catalog, n, producto, fecha_inicial, fecha_final):
    def criterio(pais_1, pais_2):
        if pais_1["eficiencia"] != pais_2["eficiencia"]:
            return pais_1["eficiencia"] > pais_2["eficiencia"]
        if pais_1["amount_total"] != pais_2["amount_total"]:
            return pais_1["amount_total"] > pais_2["amount_total"]
        return pais_1["Country"] <= pais_2["Country"]

    start_time = get_time()

    pedidos_producto = mlp.get(catalog["OrdenProd"], producto)

    resumen_paises = mlp.new_map(20, 0.5)
    paises_encontrados = lt.new_list()

    total_pedidos = 0
    amount_total = 0.0
    marketing_total = 0.0

    if pedidos_producto is not None:
        for i in range(lt.size(pedidos_producto)):
            pedido = lt.get_element(pedidos_producto, i)
            fecha = pedido["Order_Date"]
            if fecha >= fecha_inicial and fecha <= fecha_final:
                total_pedidos += 1
                amount_total += pedido["Amount"]
                marketing_total += pedido["Marketing_Spend"]
```

```python
                pais = pedido["Country"]
                resumen = mlp.get(resumen_paises, pais)
                if resumen is None:
                    resumen = {
                        "Country": pais,
                        "num_pedidos": 0,
                        "amount_total": 0.0,
                        "marketing_total": 0.0,
                        "boxes_total": 0,
                        "suma_precio": 0.0,
                        "precio_promedio": 0.0,
                        "eficiencia": 0.0,
                    }
                    mlp.put(resumen_paises, pais, resumen)
                    lt.add_last(paises_encontrados, resumen)

                resumen["num_pedidos"] += 1
                resumen["amount_total"] += pedido["Amount"]
                resumen["marketing_total"] += pedido["Marketing_Spend"]
                resumen["boxes_total"] += pedido["Boxes_Shipped"]
                resumen["suma_precio"] += pedido["Price_per_Box"]

    total_paises = lt.size(paises_encontrados)

    ranking = lt.new_list()
    for i in range(total_paises):
        resumen = lt.get_element(paises_encontrados, i)
        resumen["precio_promedio"] = resumen["suma_precio"] / resumen["num_pedidos"]
        if resumen["marketing_total"] > 0:
            resumen["eficiencia"] = resumen["amount_total"] / resumen["marketing_total"]
            lt.add_last(ranking, resumen)

    lt.merge_sort(ranking, criterio)
```

```
    top_paises = lt.new_list()
    limite = lt.size(ranking)
    if n < limite:
        limite = n
    for i in range(limite):
        lt.add_last(top_paises, lt.get_element(ranking, i))

    end_time = get_time()
    tiempo = delta_time(start_time, end_time)
    return (tiempo, total_pedidos, total_paises,
            amount_total, marketing_total, top_paises)
```

Identifica los N países con mayor eficiencia de mercadeo —suma (Amount) / suma(Marketing_Spend)— para un producto dentro de un rango de fechas. Usa dos tablas de hash Linear Probing: OrdenProd, construida en la carga, con llave Product y valor la lista de sus pedidos; y una auxiliar creada en la consulta, con llave Country y valor un registro acumulador por país. Se recorre una sola vez la lista del producto, se verifica el rango comparando las fechas como cadenas (el formato YYYY-MM-DD es lexicográficamente ordenable) y se actualiza el país en O(1); un array_list paralelo guarda los países encontrados para no barrer toda la tabla al final. Luego se calculan precio promedio y eficiencia, se descartan los países con Marketing_Spend total en cero, se ordena con Merge Sort y se toman los primeros N, o todos si N los supera.

Entrada	catalog, n, producto, fecha_inicial, fecha_final
Salidas	tiempo, total_pedidos, total_paises, amount_total, marketing_total, top_paises
Implementado (Sí/No)	Implementado por Juan Rojas, Katy Terán y Juan José Rincón

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el producto en la tabla de hash OrdenProd	O(1) promedio
Crear la tabla de hash auxiliar de países	O(1)
Recorrer los p pedidos del producto y comparar cada fecha contra el rango	O(p)
Por cada uno de los f pedidos dentro del rango: buscar e insertar el país en la tabla auxiliar y acumular sus seis valores	O(1) por pedido entonces es O(f)

Recorrer los c países para calcular el precio promedio y la eficiencia, descartando los de Marketing_Spend igual a cero	O(c)
Ordenar los c países con Merge Sort	O(c log c)
Extraer los primeros N países del ranking (N ≤ c)	O(c)
TOTAL	***O(p + c log c)***

Análisis

El costo está dominado por el recorrido de los pedidos del producto, O(p), porque con solo cinco países el ordenamiento es en la práctica constante. La ganancia depende de la consulta: 70% Dark Bar recorre el 32% del archivo y Milk Classic Bar en seis meses menos del 2%. Se usó Linear Probing en ambas tablas por sus factores de carga bajísimos (0.06 y 0.12), donde el sondeo resuelve casi siempre en la primera posición; la tabla auxiliar de países evita una segunda pasada al agregar todo en el mismo recorrido.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<2>>

Descripción

```python
def req_2(catalog, pais, min_cajas, max_cajas):
    """
    Retorna el resultado del requerimiento 2
    """
    start_time = get_time()

    pedidos_pais = msc.get(catalog["OrdenPaisSC"], pais)
    pedidos_filtrados = lt.new_list()
    frecuencias = msc.new_map(100, 1)

    total_pedidos = 0
    productos_diferentes = 0
    suma_marketing = 0
    suma_amount = 0
    producto_frecuente = None
    frecuencia_maxima = 0

    if pedidos_pais is not None:
        for i in range(lt.size(pedidos_pais)):
            pedido = lt.get_element(pedidos_pais, i)
            cajas = pedido["Boxes_Shipped"]

            if min_cajas <= cajas <= max_cajas:
                lt.add_last(pedidos_filtrados, pedido)
                total_pedidos += 1

                suma_marketing += pedido["Marketing_Spend"]
                suma_amount += pedido["Amount"]

                producto = pedido["Product"]
                cantidad = msc.get(frecuencias, producto)

                if cantidad is None:
                    cantidad = 1
                    productos_diferentes += 1
                else:
                    cantidad += 1

                msc.put(frecuencias, producto, cantidad)

                if cantidad > frecuencia_maxima:
                    frecuencia_maxima = cantidad
                    producto_frecuente = producto

                elif cantidad == frecuencia_maxima:
                    if producto_frecuente is None or producto < producto_frecuente:
                        producto_frecuente = producto

    promedio_marketing = 0
    promedio_amount = 0

    if total_pedidos > 0:
        promedio_marketing = suma_marketing / total_pedidos
        promedio_amount = suma_amount / total_pedidos
```

```
def comparar(pedido1, pedido2):
    cajas1 = pedido1["Boxes_Shipped"]
    cajas2 = pedido2["Boxes_Shipped"]

    if cajas1 != cajas2:
        return cajas1 < cajas2

    fecha1 = pedido1["Order_Date"]
    fecha2 = pedido2["Order_Date"]

    if fecha1 != fecha2:
        return fecha1 > fecha2

    return pedido1["Order_ID"] <= pedido2["Order_ID"]

lt.merge_sort(pedidos_filtrados, comparar)

pedidos_mostrar = lt.new_list()

if total_pedidos > 20:
    for i in range(10):
        pedido = lt.get_element(pedidos_filtrados, i)
        lt.add_last(pedidos_mostrar, pedido)

    for i in range(total_pedidos - 10, total_pedidos):
        pedido = lt.get_element(pedidos_filtrados, i)
        lt.add_last(pedidos_mostrar, pedido)
else:
    pedidos_mostrar = pedidos_filtrados

end_time = get_time()
tiempo = delta_time(start_time, end_time)

return {
    "tiempo": tiempo,
    "total_pedidos": total_pedidos,
    "productos_diferentes": productos_diferentes,
    "promedio_marketing": promedio_marketing,
    "promedio_amount": promedio_amount,
    "producto_frecuente": producto_frecuente,
    "frecuencia_maxima": frecuencia_maxima,
    "pedidos": pedidos_mostrar
}
```

Consulta los pedidos de un país cuyo número de cajas enviadas (Boxes_Shipped) se encuentre dentro de un rango inclusivo. Durante la carga se construye la tabla hash OrdenPaisSC (Separate Chaining), que utiliza el país como llave y almacena un array_list con sus pedidos, permitiendo acceder a ellos sin recorrer toda la lista principal.

La consulta filtra los registros y calcula la cantidad de pedidos, el número de productos distintos, los promedios de inversión en mercadeo y monto, y el producto más frecuente. Para contar las frecuencias se utiliza otra tabla hash de Separate Chaining, donde la llave es el nombre del producto y el valor es su cantidad de apariciones.

Los pedidos filtrados se ordenan con Merge Sort por cajas ascendentes, fecha descendente y Order_ID ascendente. Si se obtienen más de 20 pedidos, se presentan los primeros diez y los últimos diez.

Entrada	catalog, pais, min_cajas, max_cajas
Salidas	tiempo, total_pedidos, productos_diferentes, promedio_marketing, promedio_amount, producto_frecuente, frecuencia_maxima, pedidos.
Implementado (Sí/No)	Implementado por Katy Teran

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el país en OrdenPaisSC mediante msc.get (Separate Chaining)	O(1) promedio
Recorrer los p pedidos del país y filtrar por rango de cajas	O(p)
Consultar y actualizar frecuencias de productos en una tabla hash	O(1) promedio por pedido
Calcular los promedios con los acumuladores	O(1)
Ordenar los k pedidos que cumplen con Merge Sort	O(k log k)
Construir la lista de presentación (hasta 20 pedidos)	O(1)
TOTAL	***O(p + k log k)***

Análisis

El uso de OrdenPaisSC me permite reducir la cantidad de pedidos que debo revisar, ya que la búsqueda en la tabla hash tiene un costo O(1) promedio y solo necesito recorrer los p pedidos del país seleccionado. Durante ese recorrido también actualizo las frecuencias de los productos mediante otra tabla hash, evitando hacer búsquedas adicionales.

El mayor costo puede estar en el ordenamiento con Merge Sort, cuya complejidad es O(k log k), donde k representa los pedidos que cumplen el filtro. Por eso, la complejidad temporal promedio de mi requerimiento es O(p + k log k).

Aunque construir el índice por país durante la carga requiere O(n) en promedio, este proceso se realiza previamente y no afecta el tiempo medido de la consulta. La carga completa tiene un costo O(n log n) debido al ordenamiento general. Considero que esta implementación es eficiente porque aprovecha las tablas hash para reducir los recorridos y solo ordena los pedidos que cumplen las condiciones.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<3>>

Descripción

```python
def req_3(catalog, fechaPedido, valor_min, valor_max):
    """
    Retorna el resultado del requerimiento 3 utilizando los TADs del proyecto
    y merge_sort para el ordenamiento múltiple.
    """
    start_time = get_time()

    fecha = mlp.get(catalog["OrdenFecha"], fechaPedido)
    count = 0
    amount__total = 0
    marketing_spend_total = 0
    aporte_comercial_total = 0

    filtrados = lt.new_list()

    if fecha is not None:
        for pos in range(lt.size(fecha)):
            orden = lt.get_element(fecha, pos)

            if valor_min <= orden["Amount"] <= valor_max:
                count += 1
                amount__total += orden["Amount"]
                marketing_spend_total += orden["Marketing_Spend"]

                aporte_comercial = orden["Amount"] - orden["Marketing_Spend"]
                aporte_comercial_total += aporte_comercial
                orden["Aporte_Comercial"] = aporte_comercial

                # Agregamos a la lista de filtrados
                lt.add_last(filtrados, orden)

    def sort_criteria(orden1, orden2):
        amount1 = float(orden1["Amount"])
        amount2 = float(orden2["Amount"])
        if amount1 != amount2:
            return amount1 > amount2

        ap1 = float(orden1["Aporte_Comercial"])
        ap2 = float(orden2["Aporte_Comercial"])
        if ap1 != ap2:
            return ap1 > ap2

        return orden1["Order_ID"] < orden2["Order_ID"]

    lt.merge_sort(filtrados, sort_criteria)

    resultado_final = lt.new_list()
    total_filtrados = lt.size(filtrados)

    if total_filtrados <= 20:
        resultado_final = filtrados
    else:
        for i in range(10):
            elem = lt.get_element(filtrados, i)
            lt.add_last(resultado_final, elem)

        inicio_ultimos = total_filtrados - 10
        for i in range(inicio_ultimos, total_filtrados):
            elem = lt.get_element(filtrados, i)
            lt.add_last(resultado_final, elem)

    end_time = get_time()
    time = delta_time(start_time, end_time)

    return time, count, amount__total, marketing_spend_total, aporte_comercial_total, resultado_final
```

Listar los pedidos en una fecha específica cuyo costo total (Amount) se encuentre dentro de un rango determinado. Durante la carga se construye la tabla hash OrdenFecha, que utiliza la fecha del pedido como llave y almacena un array_list con sus respectivos registros, permitiendo acceder a ellos de forma directa sin recorrer toda la lista principal de pedidos.

La consulta filtra los registros de esa fecha según un rango inclusivo de costo total (valor_min y valor_max). Durante el filtrado, se calcula el número total de pedidos que cumplen la condición, el monto total recaudado (Amount), la inversión total en mercadeo (Marketing_Spend) y el aporte comercial total (definido como Amount - Marketing_Spend).

Los pedidos filtrados se ordenan utilizando merge_sort bajo un criterio múltiple: primero por Amount de manera descendente, en caso de empate por Aporte_Comercial de manera descendente, y si persiste el empate, por Order_ID de manera ascendente. Si la lista resultante contiene más de 20 registros, se presentan únicamente los primeros 10 y los últimos 10.

Entrada	catalog, fechaPedido, valor_min, valor_max
Salidas	time, count, amount__total, marketing_spend_total, aporte_comercial_total, resultado_final
Implementado (Sí/No)	Implementado por Juan José Rincon Santos

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar la fecha en OrdenFecha mediante mlp.get (Linear_Probing)	O(1) promedio
Recorrer los p pedidos de la fecha y filtrar por rango de costo	O(p)
Calcular acumuladores, aporte comercial y agregar a la lista	O(1) promedio por pedido
Ordenar los k pedidos que cumplen con merge_sort	O(k log k)
Construir la lista de presentación (primeros 10 y últimos 10)	O(1)
TOTAL	***O(p + k log k)***

Análisis

El uso de la tabla hash OrdenFecha permite optimizar de manera significativa la búsqueda, reduciendo el conjunto de datos a revisar mediante un costo de acceso O(1) promedio, lo que evita recorrer la lista general y limita el recorrido únicamente a los p pedidos asociados a la fecha solicitada.

Durante dicho recorrido, se evalúa el filtro de costo, se actualizan los acumuladores estadísticos y se calcula el aporte comercial de cada orden en tiempo constante por elemento.

El mayor costo temporal recae sobre el proceso de ordenamiento múltiple ejecutado mediante merge_sort, cuya complejidad teórica corresponde a O(k log k), donde k representa la cantidad de pedidos que satisfacen las condiciones del rango. Por consiguiente, la complejidad temporal total del requerimiento se define como O(p + k log k).

La construcción de las estructuras de indexación se realiza previamente durante la fase de carga de datos (Load_data), por lo que no interfiere de forma directa con el tiempo medido durante la ejecución de la consulta. Esta implementación resulta eficiente al aprovechar la indexación hash y aplicar el algoritmo de ordenamiento únicamente sobre el subconjunto de datos filtrados.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<4>>

Descripción

```python
def req_4(catalog, country, product):
    """
    Retorna el resultado del requerimiento 4
    """

    def es_mejor_pedido(orden1, orden2):
        """
        Compara dos pedidos según los criterios del requerimiento.
        """
        if float(orden1["Amount"]) > float(orden2["Amount"]):
            return True

        elif float(orden1["Amount"]) < float(orden2["Amount"]):
            return False

        if float(orden1["Marketing_Spend"]) < float(orden2["Marketing_Spend"]):
            return True

        elif float(orden1["Marketing_Spend"]) > float(orden2["Marketing_Spend"]):
            return False

        return orden1["Order_ID"] < orden2["Order_ID"]

    start_time = get_time()

    llave = product + "-" + country
    orders = msc.get(catalog["OrdenProdPais"], llave)

    count = 0
    prom_box = 0
    prom_disc = 0
    prom_mark = 0
    prom_box_ship = 0

    orden1 = None
    orden2 = None

    if orders is not None:
        for pos in range(0, lt.size(orders)):
            orden = lt.get_element(orders, pos)

            count += 1
            prom_box += float(orden["Price_per_Box"])
            prom_disc += float(orden["Discount_Pct"])
            prom_mark += float(orden["Marketing_Spend"])
            prom_box_ship += int(orden["Boxes_Shipped"])

            if orden1 is None:
                orden1 = orden

            elif es_mejor_pedido(orden, orden1):
                orden2 = orden1
                orden1 = orden

            elif orden2 is None:
                orden2 = orden

            elif es_mejor_pedido(orden, orden2):
                orden2 = orden
```

```
    if count == 0:
        return None

    prom_box = prom_box / count
    prom_disc = prom_disc / count
    prom_mark = prom_mark / count
    prom_box_ship = prom_box_ship / count

    amount1 = {
        "ID": orden1["Order_ID"],
        "channel": orden1["Channel"],
        "date": orden1["Order_Date"],
        "box_ship": orden1["Boxes_Shipped"],
        "amount": orden1["Amount"]
    }

    amount2 = None

    if orden2 is not None:
        amount2 = {
            "ID": orden2["Order_ID"],
            "channel": orden2["Channel"],
            "date": orden2["Order_Date"],
            "box_ship": orden2["Boxes_Shipped"],
            "amount": orden2["Amount"]
        }

    end_time = get_time()
    time = delta_time(start_time, end_time)

    return (
        time,
        count,
        prom_box,
        prom_disc,
        prom_mark,
        prom_box_ship,
        amount1,
        amount2
    )
```

Consulta los pedidos de una combinación específica de producto y país. Durante la carga se construye la tabla hash OrdenProdPais, implementada con Separate Chaining, que utiliza la combinación Product-Country como llave y guarda un array_list con los pedidos correspondientes. Así podemos consultar directamente esa combinación sin recorrer la lista principal.

Con los pedidos encontrados calculamos la cantidad total y los promedios de Price_per_Box, Discount_Pct, Marketing_Spend y Boxes_Shipped. También seleccionamos los dos pedidos de mayor Amount; cuando hay empate, tiene prioridad el de menor Marketing_Spend y, si persiste, el de menor Order_ID. De cada pedido seleccionado se presentan su identificador, canal, fecha, cajas enviadas y monto.

Entrada	catalog, country, product
Salidas	time, count, prom_box, prom_disc, prom_mark, prom_box_ship, amount1, amount2, retorna None si no hay coincidencias.
Implementado (Sí/No)	Implementado por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Construir la llave Product-Country	O(1)
Consultar OrdenProdPais mediante msc.get (Separate Chaining)	O(1) promedio
Recorrer los k pedidos de la combinación y acumular valores	O(k)
Comparar y actualizar los dos pedidos de mayor Amount	O(1) por pedido
Calcular los cuatro promedios y construir las salidas	O(1)
TOTAL	***O(1+k) promedio***

Análisis

Como utilizamos la tabla hash OrdenProdPais, la consulta comienza con una búsqueda O(1) promedio y únicamente recorre los k pedidos del producto y país seleccionados. En ese mismo recorrido acumulamos las sumas para los promedios y mantenemos los dos pedidos con mayor Amount. Cada comparación tiene costo constante, por lo que no hace falta ordenar todos los resultados.

Por esta razón, la complejidad temporal promedio del requerimiento es O(1 + k). Es una mejora frente al Requerimiento 4 del Reto 1, que recorre los n pedidos de la lista principal y tiene costo O(n). La diferencia es especialmente útil cuando la combinación consultada representa una parte pequeña del archivo.

La tabla OrdenProdPais se construye durante load_data: por cada pedido se consulta o inserta su llave y se agrega a la lista asociada, con costo O(1) promedio o amortizado por registro. Por eso, construir este índice cuesta O(n) promedio. La carga completa alcanza O(n log n), debido al Merge Sort utilizado para ordenar todos los pedidos del resumen de carga; este tiempo no se incluye en la medición del requerimiento.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<5>>

Descripción

```python
def req_5(catalog, n, pais, fecha_inicio, fecha_fin):
    """
    Retorna el resultado del requerimiento 5
    """
    start_time = get_time()

    pedidos_pais = msc.get(catalog["OrdenPaisSC"], pais)
    productos = msc.new_map(100, 1)
    lista_productos = lt.new_list()

    total_pedidos = 0
    recaudo_total = 0

    if pedidos_pais is not None:
        for i in range(lt.size(pedidos_pais)):
            pedido = lt.get_element(pedidos_pais, i)
            fecha = pedido["Order_Date"]

            if fecha_inicio <= fecha <= fecha_fin:
                total_pedidos += 1

                nombre = pedido["Product"]
                amount = pedido["Amount"]
                cajas = pedido["Boxes_Shipped"]
                precio = pedido["Price_per_Box"]
                descuento = pedido["Discount_Pct"]
                marketing = pedido["Marketing_Spend"]

                recaudo_total += amount

                producto = msc.get(productos, nombre)

                if producto is None:
                    producto = {
                        "Product": nombre,
                        "numero_pedidos": 0,
                        "total_cajas": 0,
                        "recaudo_total": 0,
                        "suma_precios": 0,
                        "suma_descuentos": 0,
                        "marketing_total": 0
                    }

                    msc.put(productos, nombre, producto)
                    lt.add_last(lista_productos, producto)

                producto["numero_pedidos"] += 1
                producto["total_cajas"] += cajas
                producto["recaudo_total"] += amount
                producto["suma_precios"] += precio
                producto["suma_descuentos"] += descuento
                producto["marketing_total"] += marketing

    total_productos = lt.size(lista_productos)
```

```
    for i in range(total_productos):
        producto = lt.get_element(lista_productos, i)
        cantidad = producto["numero_pedidos"]

        producto["precio_promedio"] = producto["suma_precios"] / cantidad
        producto["descuento_promedio"] = producto["suma_descuentos"] / cantidad

    def comparar(producto1, producto2):
        recaudo1 = producto1["recaudo_total"]
        recaudo2 = producto2["recaudo_total"]

        if recaudo1 != recaudo2:
            return recaudo1 > recaudo2

        cajas1 = producto1["total_cajas"]
        cajas2 = producto2["total_cajas"]

        if cajas1 != cajas2:
            return cajas1 > cajas2

        return producto1["Product"] <= producto2["Product"]

    lt.merge_sort(lista_productos, comparar)

    top_productos = lt.new_list()

    cantidad_mostrar = n

    if cantidad_mostrar > total_productos:
        cantidad_mostrar = total_productos

    if cantidad_mostrar < 0:
        cantidad_mostrar = 0

    for i in range(cantidad_mostrar):
        producto = lt.get_element(lista_productos, i)

        resumen = {
            "Product": producto["Product"],
            "numero_pedidos": producto["numero_pedidos"],
            "total_cajas": producto["total_cajas"],
            "recaudo_total": producto["recaudo_total"],
            "precio_promedio": producto["precio_promedio"],
            "descuento_promedio": producto["descuento_promedio"],
            "marketing_total": producto["marketing_total"]
        }

        lt.add_last(top_productos, resumen)

    end_time = get_time()
    tiempo = delta_time(start_time, end_time)

    return {
        "tiempo": tiempo,
        "total_pedidos": total_pedidos,
        "total_productos": total_productos,
        "recaudo_total": recaudo_total,
        "top_productos": top_productos
    }
```

Identifica los N productos con mayor recaudo para un país dentro de un rango inclusivo de fechas. Para esto, se consulta la tabla hash OrdenPaisSC (Separate Chaining), construida durante la carga, cuya llave es el país y cuyo valor es un array_list con sus pedidos. Esto permite acceder directamente a los registros del país seleccionado.

Después se recorren esos pedidos, se filtran por fecha y se agrupan por producto mediante otra tabla hash de Separate Chaining llamada productos. En esta tabla, la llave es el nombre del producto y el valor es un diccionario donde se acumulan la cantidad de pedidos, cajas enviadas, recaudo, precios por caja, descuentos y gasto de marketing.

Finalmente, se calculan los promedios y se ordenan los productos utilizando Merge Sort por recaudo descendente, cajas descendentes y nombre alfabético ascendente. Se presentan los primeros N productos del ranking.

Entrada	catalog, n, pais, fecha_inicio, fecha_fin
Salidas	tiempo, total_pedidos, total_productos, recaudo_total, top_productos (con número de pedidos, cajas, recaudo, precio y descuento promedio y marketing total de cada producto).
Implementado (Sí/No)	Implementado por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Buscar el país en OrdenPaisSC mediante msc.get (Separate Chaining)	O(1) promedio
Recorrer los p pedidos del país y comprobar el rango de fechas	O(p)
Agrupar y acumular datos mediante la tabla hash productos	O(1) promedio por pedido
Calcular promedios de los d productos diferentes	O(d)
Ordenar los d productos con Merge Sort	O(d log d)
Construir la lista de los primeros min(N, d) productos	O(min(N,d))
TOTAL	***O(p + d log d)***

Análisis

Al utilizar OrdenPaisSC, podemos acceder directamente a los pedidos del país solicitado, con un costo de búsqueda O(1) promedio. Esto nos evita recorrer todos los registros del archivo y limita el procesamiento a los p pedidos de ese país.

La tabla hash auxiliar productos también nos permite acumular las estadísticas de cada producto en tiempo O(1) promedio por operación. Después, el ordenamiento con Merge Sort tiene un costo O(d log d), donde d es la cantidad de productos diferentes encontrados. Por lo tanto, la complejidad temporal promedio de nuestro requerimiento es O(p + d log d).

La construcción del índice por país tiene complejidad O(n) promedio durante la carga, mientras que la carga completa alcanza O(n log n) por el ordenamiento general. Como estas operaciones se realizan antes de la consulta, no se incluyen en su tiempo de ejecución. La ventaja de nuestra solución es que agrupamos los pedidos mediante hash y ordenamos únicamente los productos diferentes, lo que reduce el trabajo necesario para obtener el Top N.